OSSにおけるレガシー関数の到達可能性を考慮したソフトウェアリスク特定手法の提案

永井 ひとみ[1,a)]　大江 一輝[2]　葛野 弘樹[2]　和泉 諭[3]　瀧田 愼[2]　白石 善明[2]

概要：ソフトウェアシステムでは多様なオープンソースソフトウェア（OSS）を利用して運用している．ソフトウェアシステムで利用される OSS に脆弱性が存在する場合，脆弱性の影響が OSS 間の依存関係を介して他の OSS にまで拡大する可能性がある．既存の脆弱性影響範囲を特定する手法は，脆弱な関数の呼出し関係に着目しており，ソフトウェアの改善や運用の過程で発生するレガシー関数を含む長期間更新されていないソースコードや，サポートが終了した OSS との依存関係を捉えられない課題がある．本研究では，新たなソフトウェアリスクとして，ソフトウェアシステムで利用される OSS に対して，レガシー関数を考慮した，脆弱性に起因する外部からの攻撃の影響が広範囲に及ぶリスクを検討する．特に，レガシー関数が OSS に多数存在する場合，継続的な保守が行われておらず，脆弱性が残存している可能性があり，ソフトウェアリスクにつながる．ソフトウェアリスク特定のため，既存の脆弱性影響範囲の特定手法を拡張し，脆弱性情報，依存関係および被依存関係を有し，End of Life（EOL）と判定された OSS，ならびに新たなソフトウェアセキュリティ情報としてレガシー情報を用いることで，評価項目の該当数から，OSS の依存関係と被依存関係に基づいた関数粒度での到達可能性を考慮したソフトウェアリスクを特定する手法を提案する．評価では，解析対象を C 言語ソースファイルに限定し，脆弱性情報と OSS パッケージから，脆弱性，EOL と判定された OSS，脆弱性影響範囲，およびレガシー関数への到達可能性を特定し，ソフトウェアリスクを 3 段階で判定可能なことを確認した．

A Proposal for Software Risk Identification Method Considering the Reachability of Legacy Function in OSS

Hitomi Nagai[1,a)]　Kazuki Oe[2]　Hiroki Kuzuno[2]　Satoru Izumi[3]　Makoto Takita[2]
Yoshiaki Shiraishi[2]

Abstract: Software systems use a wide variety of open-source software (OSS). When a vulnerability exists in OSS used in a software system, its impact may propagate to other OSS through dependency relationships. Existing methods for identifying vulnerability impact scopes focus on call relationships involving vulnerable functions, but do not consider long-unmodified source code containing legacy functions or dependencies on OSS whose support has ended. In this study, we consider a new type of software risk in which the impact of external attacks exploiting vulnerabilities spreads widely across a software system, taking legacy functions into account. In particular, when a large number of legacy functions exist in OSS, continuous maintenance may not have been sufficiently performed, and vulnerabilities may remain, potentially leading to software risk. To identify such software risk, we extend an existing method for identifying vulnerability impact scopes and use vulnerability information, OSS in dependency and reverse-dependency relationships that are determined to have reached End of Life (EOL), and legacy information as additional software security information. Based on the number of satisfied evaluation criteria, the proposed method identifies software risk while considering function-level reachability based on OSS dependencies and reverse dependencies.In the evaluation, we limited the analysis to C source files and confirmed that the proposed method can identify vulnerabilities, OSS determined to have reached EOL, vulnerability impact scopes, and reachability to legacy functions from vulnerability information and OSS packages, and can classify software risk into three levels.

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

1. はじめに

ソフトウェア開発の迅速化やコストの削減を目的として，オープンソースソフトウェア（OSS）の活用が拡大している．ソフトウェアシステムの多くは，独自のソースコードだけでなく，OSS を利用して開発され，運用されている．ソフトウェアシステムで利用される OSS に脆弱性が発見された場合，攻撃による影響が関数呼出し等の依存関係を介して他の OSS に拡大するリスクがある．

OSS に対する脆弱性影響範囲の特定手法として，バイナリやソースコードから関数呼出しグラフを生成し，OSS の依存関係から脆弱性影響範囲を把握する手法が提案されている [1], [2], [3]．しかし，既存研究では，脆弱性による影響範囲を把握可能である一方，ソフトウェアの改善や運用の過程で発生するレガシー関数の影響範囲は考慮されない．

レガシー関数は，一定期間修正がなく，非推奨の古いライブラリやメソッドを利用する関数である [4]．レガシー関数が他の OSS やライブラリからの関数呼出し経路を有する場合，残存する脆弱性の悪用可能性があり，レガシー関数を考慮したソフトウェアリスクの検討が求められる．

攻撃による影響に対し，ソフトウェアシステムで利用される OSS の脆弱性に加え，レガシー関数を考慮し，ソフトウェアシステムの広範囲に及ぶソフトウェアリスクを特定するには，以下の課題が存在する．

課題：レガシー関数と EOL の把握

既存研究では，一定期間更新されていないレガシー関数の把握は考慮されていない．OSS において，継続的な保守が行われておらず，End of Life（EOL）に近づく場合，OSS におけるレガシー関数に脆弱性が残存する可能性がある．ソフトウェアリスクの特定には，レガシー関数と EOL の把握が課題となる．

本研究は，課題としたソフトウェアリスクの特定のために，既存の脆弱性影響範囲の特定手法 [1] を拡張し，OSS の依存関係と被依存関係に基づいたレガシー関数への到達可能性の特定に加え，依存関係にある OSS の EOL を考慮したソフトウェアリスクの特定手法を提案する．

OSS 開発者による OSS のサポートが終了し，EOL と判定された場合，継続的なメンテナンスは行われなくなる．OSS のレガシー関数が脆弱性を含んだ場合，ソフトウェアシステムにおける広範囲に及ぶソフトウェアリスクとなる．レガシー関数の到達可能性を特定するため，他のパッケージやライブラリから呼出し経路を有する関数をエントリ関数とし，エントリ関数からレガシー関数の呼出し経路の有無を特定する．提案するソフトウェアリスクの特定手法の概要を図 1 に示す．提案手法では，脆弱性が報告された OSS，およびソフトウェア構成情報である Software Bill of Materials（SBOM）を基に，脆弱性，脆弱性影響範囲，EOL と判定された OSS，およびレガシー関数への到達可能性の評価項目の該当数を計算し，ソフトウェアリスクを高，中，および低の 3 段階で判定する．

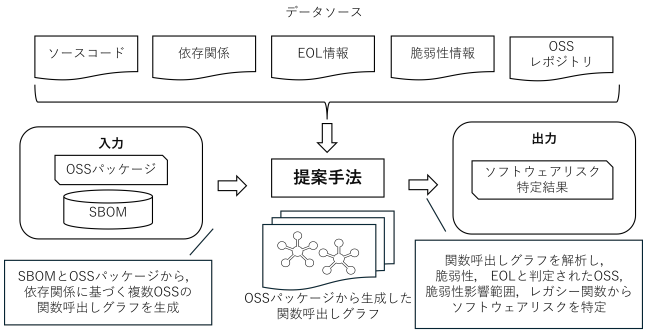


図 1 提案手法の概要

本研究での貢献は以下の通りである．

- レガシー関数を考慮した OSS のソフトウェアリスクを特定するため，ソースコードの静的解析から生成した関数呼出しグラフを基に，SBOM，依存関係および被依存関係を利用し，脆弱性と脆弱性影響範囲，レガシー関数の到達可能性，および EOL と判定された OSS を特定する手法を設計，実装した．提案手法を Linux ディストリビューションへ適用し機能を検証した．
- 提案手法の評価にて，複数の OSS に対し，既知の脆弱性影響範囲，到達可能なレガシー関数，および EOL と判定された OSS を特定可能であることを確認した．また，OSS のソースコード解析およびソフトウェアリスク特定に要する計算時間を計測した．

2. 背景

2.1 関数呼出しグラフ

関数呼出しグラフは，ソースコード中の関数を頂点，関数間の呼出し関係を辺として，ソースコードの静的解析により生成される．関数呼出しには，直接呼出しと間接呼出しが存在する．直接呼出しはソースコード上で関数の名称を用いた呼出しであり，間接呼出しはソースコード実行時に呼出先の関数が決定される呼出しである．

関数呼出しグラフ上で，他のパッケージやライブラリから呼出し経路を有する関数をエントリ関数とする．エントリ関数から到達可能な関数は，実行時に呼び出される可能性がある関数である．

2.2 EOL とレガシー情報

EOL は，製品やサービスのライフサイクルが終わり，

[1] 神戸大学 工学部 電気電子工学科
Department of Electrical and Electronic Engineering, Faculty of Engineering, Kobe University
[2] 神戸大学 大学院工学研究科 電気電子工学専攻
Department of Electrical and Electronic Engineering, Graduate School of Engineering, Kobe University
[3] 仙台高等専門学校
National Institute of Technology, Sendai College
[a)] hitomi.nagai@gsuite.kobe-u.ac.jp

© 2026 Information Processing Society of Japan

開発者によるサポートが終了したことを示す [5]．OSS が EOL と判定された場合，継続的なメンテナンスは行われず，脆弱性に対して修正パッチは配布されない．

EOL と判定された OSS がソフトウェアシステムに利用された場合，脆弱性の影響は関数呼出し等の依存関係を介して，ソフトウェアシステム内の他の OSS に及ぶ危険性があり，ソフトウェアリスクとなる．

レガシー関数は，一定期間修正が行われておらず，非推奨となる古いライブラリやメソッドが使用されている可能性のある関数である [4]．ソフトウェアの開発および保守が継続される場合でも，特定の関数はリファクタリングや機能追加対象から外れ，メンテナンス停滞とみなされる．

ソフトウェアにおける全ての関数は時間の経過に伴いレガシー関数となるリスクがあり，既知の脆弱な実装パターンを含んだまま修正されずに残存する可能性がある．レポジトリ管理されている OSS の場合，レガシー情報として，関数の最終更新日時を利用可能である．レガシー関数が OSS 内に大量に存在する場合，継続的な保守が行われておらず，EOL に近づいているとみなすことができ，ソフトウェアリスクとなる．

2.3 脆弱性情報と OSS 依存関係情報

脆弱性情報には Common Vulnerabilities and Exposures（CVE）および Common Vulnerability Scoring System（CVSS）が含まれる．脆弱性は攻撃に利用可能な実装不備であり，CVE として，登録済みの脆弱性の識別に用いられる一意の番号が割り当てられ，NVD（National Vulnerability Database）において管理，公開されている [6]．CVE のリファレンス情報により，影響を受けるバージョンおよび修正パッチ情報を取得可能である．

CVSS は，CVE の深刻度を定量化する評価指標であり，0.0 から 10.0 のスコアで表される [7]． スコアの高低で深刻度を表し，スコア 7.0 以上は重要な脆弱性とされる [8]．

OSS 依存関係情報は，OSS 間における実行時の依存関係および被依存関係であり，Linux ディストリビューションでは，パッケージ管理システムを用いて OSS を管理し，依存関係を解決している．

Debian GNU/Linux では，deb パッケージを用いて，OSS を管理し，配布している [9]．deb パッケージは，ソースパッケージ名，対応するアーキテクチャ，OSS のレポジトリ情報，および依存関係の情報を含む．パッケージのソースコードを取得し，依存関係を再帰的に探索可能である．

3. 想定モデル

本研究における想定モデルとして，ソフトウェアリスクの特定を行う範囲は，脆弱性が報告された OSS と依存関係および被依存関係を有する OSS とする．

3.1 想定環境

- OSS：Linux ディストリビューションにて，パッケージ管理された OSS とし，パッケージ情報に依存関係情報が記述されているとする．
- 脆弱性が報告された OSS：ソースコードの特定箇所に脆弱性を含むことが報告され，CVE に登録されているとする．脆弱性修正に対応するコミット情報は，CVE のリファレンス情報から取得可能とする．
- 依存関係および被依存関係を有する OSS：動作にあたり，ソフトウェアリスクを有する OSS を利用し，依存関係を有するとしてパッケージ管理されている OSS とする．なお，被依存関係情報は，パッケージ管理された OSS の依存関係情報から探索可能とする．
- エントリ関数：main 関数，および他のパッケージやライブラリから到達可能な関数とする．

3.2 想定シナリオ

提案手法にて想定するソフトウェアリスクの特定が必要となるシナリオは以下の通りである．

- 脆弱性情報：セキュリティ研究者等により特定の OSS に対する脆弱性が発見，報告され，CVE が発行されているとする．
- 脆弱性影響範囲：各 OSS の依存関係および被依存関係は把握可能であるが，ソースコードを基にした関数粒度での脆弱性影響範囲は不明である．
- レガシー関数：各 OSS のソースコードに含まれる関数単位の最終更新日時は不明であり，レガシー関数への到達可能性は把握できないとする．
- EOL 情報：依存関係および被依存関係を有する OSS が EOL と判定されているかは不明であるとする．

4. 提案手法

4.1 要件

提案手法では，脆弱性が報告された OSS，依存関係および被依存関係を有する OSS に対し，ソフトウェアリスク特定のため，設計にて次の要件を満たすことを目指した．

要件： ソフトウェアシステムにて利用される OSS に対し，依存関係および被依存関係を有する OSS のうち，EOL と判定された OSS を特定する．また，脆弱な関数を呼び出す可能性のある関数とエントリ関数からレガシー関数への到達可能性を調査し，ソフトウェアリスクを特定する．

4.2 ソフトウェアリスク

提案手法では，表 1，表 2 に示すソフトウェアリスクの評価項目とソフトウェアリスク判定内容を用いてソフトウェアリスクを判定する．

ソフトウェアリスクは，脆弱性，脆弱性影響範囲，到達

© 2026 Information Processing Society of Japan

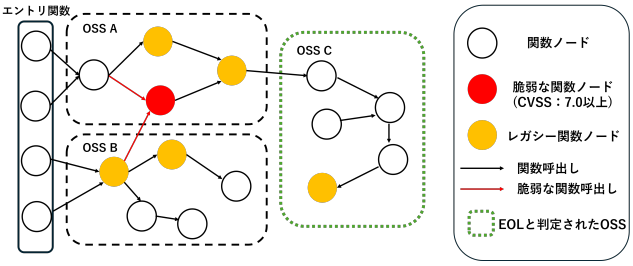


図 2　ソフトウェアリスク特定に用いる関数呼出しグラフの例

表 1　ソフトウェアリスク評価項目

評価項目	条件
脆弱性	CVSS が 7.0 以上
脆弱性影響範囲	脆弱性の影響が他パッケージへ到達する
レガシー関数	到達可能なレガシー関数の割合が基準値以上である
EOL	依存関係にある OSS に EOL と判定された OSS が存在する

表 2　ソフトウェアリスクと判定内容

評価項目該当数	ソフトウェアリスク	判定内容
3〜4	高	優先的な対策が必要
2	中	継続的な監視が必要
0〜1	低	リスク要因が限定的

可能なレガシー関数，および EOL と判定された OSS の 4 つの評価項目の条件より，評価項目該当数から 3 段階で判定を行う．表 2 より，ソフトウェアリスクは高，中，および低で表し，判定内容は優先的な対策，継続的な監視，ならびにリスク要因が限定的，の 3 段階とした．

図 2 に OSS A，B，ならびに C に対するソフトウェアリスク特定における関数呼出しグラフの例を示す．図 2 において，OSS A はソフトウェアリスクの評価項目として，CVSS が高い脆弱性，OSS B の関数から OSS A の脆弱関数へ到達する呼出し経路，レガシー関数，ならびに EOL と判定された OSS C の関数呼出し，の 4 つのソフトウェアリスク評価項目に該当するため，ソフトウェアリスクは高の優先的な対策が必要と判定される．

4.3　設計

4.3.1　設計方針

ソフトウェアリスクを特定するため，要件に対する設計において，設計方針は次の通りとした．

- 設計方針 1：脆弱性情報より，CVSS，脆弱性が報告された OSS，依存関係および被依存関係を有する OSS を特定し，EOL と判定された OSS を特定する．
- 設計方針 2：脆弱性が報告された OSS，依存関係および被依存関係を有する OSS のソースコードを収集し，静的解析を実行，関数呼出しグラフを生成し，脆弱性影響範囲と到達可能なレガシー関数を特定する．
- 設計方針 3：脆弱性の CVSS，EOL と判定された OSS，脆弱性影響範囲，および到達可能なレガシー関数に基づいて，評価項目の該当数を計算し，ソフトウェアリスクを高，中，および低の 3 段階で判定する．

4.3.2　設計概要

設計方針に基づいた提案手法の設計概要を図 3 に示す．提案手法は以下の 5 つのステップから構成される．

- Step 1：OSS 特定とソースコード取得
- Step 2：関数呼出しグラフ生成
- Step 3：EOL と判定されたの特定
- Step 4：脆弱性影響範囲特定
- Step 5：到達可能なレガシー関数特定

4.3.3　OSS 特定とソースコード取得

Step 1 では，脆弱性が報告された OSS を特定し，SBOM とパッケージ管理システムを基に，依存関係および被依存関係を有する OSS の特定とソースコード取得を行う．一連の流れを以下に示す．

- **Step 1-1**：脆弱性スキャナを利用し，脆弱性が報告された OSS のパッケージ名を取得
- **Step 1-2**：SBOM 生成機能を利用し，SBOM を作成
- **Step 1-3**：パッケージ管理システムを利用し，依存関係および被依存関係を有する OSS のパッケージ名を取得し，SBOM を用いてソフトウェアシステムに含まれるパッケージを確認し，調査範囲を決定
- **Step 1-4**：パッケージ管理システムを利用し，脆弱性が報告された OSS，依存関係および被依存関係を有する OSS のソースコードを取得

4.3.4　関数呼出しグラフの生成

Step 2 では，関数呼出しグラフを生成する．脆弱性が報告された OSS，依存関係および被依存関係を有する OSS のソースコードを静的解析し，OSS 毎の関数呼出しグラフを生成，OSS 間の関数呼出し関係を含む 1 つの関数呼出しグラフに合成する．一連の流れを以下に示す．

- **Step 2-1**：取得した OSS のソースコードに対し，静的解析を行い，提案手法で用いる OSS 毎の関数呼出しグラフを生成
  - **Step 2-1-a**： 関数名をラベルとする頂点を作成
  - **Step 2-1-b**： 関数から呼出される関数を特定し，同関数名をラベルとする頂点を作成
  - **Step 2-1-c**： 各頂点毎に呼出し関係の存在を確認し，呼出し関係が存在する場合，直接呼出しとして辺を作成
- **Step 2-2**：生成した OSS 毎の関数呼出しグラフに対し，関数名とパッケージ名を用いて OSS 間の関数呼出し関係を特定し，1 つの関数呼出しグラフとして合成
  - **Step 2-2-a**： OSS 毎の関数呼出しグラフにて，頂点の関数名を取得
  - **Step 2-2-b**： 他の OSS の関数呼出しグラフにより，取得した関数名に該当する頂点を探索し，関数名およびパッケージ名の一致を確認

© 2026 Information Processing Society of Japan

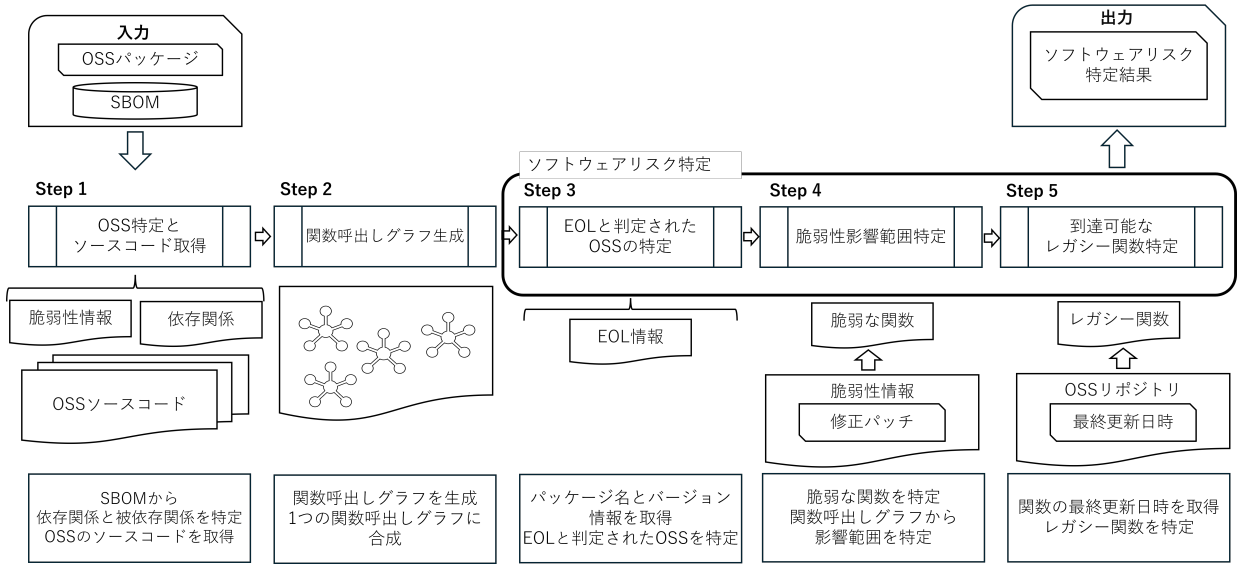


図 3 提案手法の設計概要図

– **Step 2-2-c：** 関数名およびパッケージ名が一致する場合，OSS 間の直接呼出しとして辺を作成し，接続元および接続先の頂点情報を付与

4.3.5 EOL と判定された OSS の特定

Step 3 では，EOL と判定された OSS の特定を行う．脆弱性が報告された OSS，依存関係および被依存関係を有する OSS のうち，EOL と判定された OSS を，OSS のパッケージ名とバージョン情報から特定する．一連の流れを以下に示す．

- **Step 3-1：**脆弱性が報告された OSS に加え，依存関係もしくは被依存関係を有し，SBOM に含まれる OSS を対象として，パッケージ情報から，パッケージ名とバージョン情報を特定
- **Step 3-2：**取得した OSS のパッケージ名とバージョン情報を基に，EOL 情報提供サービスと照合し，EOL と判定された OSS を特定

4.3.6 脆弱性影響範囲の特定

Step 4 では，脆弱性影響範囲を特定する．生成した関数呼出しグラフの経路探索により，特定した脆弱な関数に到達可能な関数を列挙し，脆弱性影響範囲とする．一連の流れを以下に示す．

- **Step 4-1：**脆弱性情報より，CVSS を取得，脆弱性が修正されたコミット情報の有無を確認し，コミット情報が存在した場合，ソースコードの修正情報から脆弱な関数を特定
- **Step 4-2：**OSS の関数呼出しグラフから脆弱な関数を探索
- **Step 4-3：**関数呼出しグラフ上で脆弱な関数と接続された辺に対して経路探索を実行し，経路中の頂点のラベルから，関数名を取得，脆弱性影響範囲を特定
- **Step 4-4：**脆弱性影響範囲において，脆弱性が報告された OSS 以外の OSS の関数が含まれていた場合，脆弱性の影響を受ける他パッケージとして特定

4.3.7 到達可能なレガシー関数特定

Step 5 では，到達可能なレガシー関数を特定する．生成した関数呼出しグラフの経路探索により，エントリ関数から到達可能な関数を特定し，最終更新日時を取得する．取得した最終更新日時と現在日時を比較し，レガシー関数を特定する．一連の流れを以下に示す．

- **Step 5-1：**脆弱性が報告された OSS の関数呼出しグラフにおいて，他の OSS やライブラリから到達可能なエントリ関数から到達可能な関数を特定
- **Step 5-2：**エントリ関数から到達可能な関数において，ソースコード共有プラットフォームから最終更新日時の取得を実施
- **Step 5-3：**取得した最終更新日時と現在日時との差分が一定期間を超えた関数をレガシー関数と判定（実装では，1 年と設定）
- **Step 5-4：**脆弱性が報告された OSS の全関数の総数と，レガシー関数の総数からレガシー関数が占める割合を取得

4.4 実現方式

提案手法の実現方式の想定環境は Debian GNU/Linux とした．実現方式では，脆弱性スキャナに `debsecan`[10]，SBOM 生成ツールに `debsbom`[11]，およびパッケージ管理システムに `apt`[12] を用いる．脆弱性が報告された OSS，依存関係を有する OSS，および被依存関係を有する OSS のソースコードに対するソフトウェアリスクを特定対象とする．OSS のソースコードの静的解析に `cflow`[13] を用い，解析結果から関数呼出しグラフを生成する．なお，`cflow` を用いた実現方式では関数の間接呼出しには対応しない．

4.4.1 EOL と判定された OSS の特定

実現方式における EOL と判定された OSS の特定は，脆

© 2026 Information Processing Society of Japan

弱性が報告された OSS，依存関係および被依存関係を有する OSS のパッケージ間の依存関係から行う．EOL 情報提供サービス `endoflife.date`[5] を用い，OSS のパッケージ名とバージョンから EOL と判定された OSS を特定する．

4.4.2 脆弱性影響範囲の特定

実現方式における脆弱性影響範囲の特定は，OSS の関数呼出しグラフの解析により行う．実現方式では，CVE 番号を用い，NVD から脆弱性情報を取得する．4.3.6 節に基づき，脆弱性影響範囲を特定する．

4.4.3 到達可能なレガシー関数特定

実現方式における到達可能なレガシー関数の特定は，4.3.7 節に基づき，脆弱性が報告された OSS の関数呼出しグラフに対し経路探索を行う．ソースコードの更新履歴の取得には `GitHub Blame`[14] を用い，取得した各行の最終更新日時を参照し，各関数に含まれる行の更新日時を比較する．同一関数内で取得可能な最新更新日時を，関数の最終更新日時とする．

5. 評価

5.1 評価概要と目的

提案手法の実現方式を Debian GNU/Linux にて適用し，複数の OSS におけるソフトウェアリスクの特定可否の調査，ならびにソフトウェアリスクの特定にかかる時間の測定を目的として評価した．評価項目と内容を以下に示す．

(1) ソフトウェアリスク特定の評価
脆弱性が報告された OSS に対し，ソフトウェアリスクを特定可能か確認するため，CVSS，EOL と判定された OSS，脆弱性影響範囲，および到達可能なレガシー関数を特定し，ソフトウェアリスクを評価した．

(2) ソフトウェアリスク特定に関する性能評価
脆弱性が報告された OSS に対するソフトウェアリスクの特定にかかる時間を計測した．

5.2 評価環境

5.2.1 評価用計算機とソフトウェア

提案手法の実現方式における評価用計算機として Windows 11 Pro を搭載した計算機（Intel(R)Core(TM) i5-12400，2.50GHz，x86-64，6 コア，メモリ 32GB）を利用し，仮想計算機として，VirtualBox のゲスト OS（4 コア，メモリ 8GB）を用いた．実現方式での評価対象 OS は Debian GNU/Linux 13.2.0，脆弱性スキャナに `debsecan` 0.4.20.2[10]，SBOM 作成に `debsbom` 0.6.0[11]，ソースコード静的解析に `cflow` 1.6[13] を用いた．

5.3 ソフトウェアリスク特定の評価

ソフトウェアリスク特定の評価において，評価対象 OSS は，評価環境とした SBOM に含まれる OSS のうち，C 言語で実装され，被依存数が多い上位パッケージとした．また，レガシー関数の割合の基準値を 10.0%とし，基準値以上の場合，到達可能なレガシー関数に関するリスクとした．

表 3 に提案手法によるソフトウェアリスク特定の評価結果を示す．評価対象 OSS における到達可能なレガシー関数の割合において，基準値以上は 3 種類の OSS が該当した．また，`OpenSSL` の CVE-2026-9076 および `Perl` の CVE-2026-4176 では，CVSS は 7.0 以上であり，脆弱性の影響は他パッケージへ到達し，さらに到達可能なレガシー関数の割合は基準値以上を示した．一方，`libxml2` の CVE-2026-11979 は CVSS は 1.8 と低い値であるが，脆弱性の影響が他パッケージへ到達し，到達可能なレガシー関数の割合は 16.7%を示した．

脆弱性が報告された OSS にて，依存関係と被依存関係からソフトウェアリスクの各評価項目の該当数を計算し，ソフトウェアリスクを判定可能なことを確認した．

5.4 ソフトウェアリスク特定に関する性能評価

提案手法の性能評価として，`OpenSSL3.5.6~deb13u2` の脆弱性 CVE-2026-9076 に対するソフトウェアリスクの特定にかかる計算時間を評価した．`OpenSSL` に対して依存関係を有する OSS のソースコードの解析による関数呼出しグラフの生成，依存関係の探索，EOL と判定された OSS，脆弱性影響範囲の特定，到達可能なレガシー関数の特定，ソフトウェアリスク特定結果出力時間を計測した．

実現方式における性能評価の結果を表 4 に示す．性能評価に関する情報として，解析対象パッケージ数は 3 個，解析対象ファイル数は 14,240 個，関数呼出しグラフ頂点数は 25,529 個，辺数は 57,645 個である．

表 4 から，CVE-2026-9076 の解析時間の合計は 966 秒である．最も処理時間を要したのは到達可能なレガシー関数の特定処理であり，`GitHub Blame` から最終更新履歴の取得時間を含む．続いて，関数呼出しグラフの生成，次に，脆弱性影響範囲の特定である．提案手法では，到達可能なレガシー関数の特定に処理時間を要することが判明した．

6. 考察

6.1 評価に対する考察

提案手法に対するソフトウェアリスクの特定評価において，脆弱性影響範囲に加え，到達可能なレガシー関数を考慮し，OSS に含まれるソフトウェアリスクを特定可能なことを確認した．特に，`Perl` の CVE-2026-4176 について，CISA の SSVC では，脆弱性の悪用が確認されていないが，攻撃を自動化可能であり，悪用に成功した場合の影響が大きい脆弱性と示されている [15]．提案手法においても，CVSS，脆弱性影響範囲，および到達可能なレガシー関数の割合からソフトウェアリスクは高と判定され，評価項目に基づくソフトウェアリスクを把握可能と考えられる．

性能評価より，提案手法の実現方式は，一定程度の時間

© 2026 Information Processing Society of Japan

表 3 ソフトウェアリスク特定評価

対象 OSS	バージョン	CVE	評価項目				ソフトウェアリスク（評価項目該当数）
			CVSS	他パッケージに対する影響	レガシー関数の割合[*1]	EOL	
libxml2	2.13.7+dfsg+really 2.9.14-2.1+deb13u3	CVE-2026-11979	×(1.8)	○	○(16.7%)	–	中（2）
libxml2	2.13.7+dfsg+really 2.9.14-2.1+deb13u3	CVE-2025-9714	×(6.2)	×	○(16.6%)	–	低（1）
OpenSSL	3.5.6-1 deb13u2	CVE-2026-9076	○(7.5)	○	○(65.1%)	–	高（3）
Perl	5.40.1-6	CVE-2026-4176	○(9.8)	○	○(16.5%)	×	高（3）
Node.js	20.19.2+dfsg-1+deb13u2	CVE-2025-59466	○(7.5)	–	×(2.8%)	○	中（2）
CPython	3.13.5-2+deb13u3	CVE-2026-4224	×(6.0)	–	×(4.7%)	–	低（0）
CPython	3.13.5-2+deb13u3	CVE-2026-7210	×(6.3)	×	×(4.6%)	–	低（0）
expat	2.7.1-2	CVE-2026-56408	×(6.9)	○	×(1.8%)	–	低（1）
liblzma5	5.8.1-1+deb13u1	CVE-2026-34743	×(1.7)	×	×(2.3%)	–	低（0）

○：評価条件に該当，×：評価条件に非該当，–：判定不可

表 4 提案手法における処理時間

処理項目	処理時間（sec）
ソースコード収集	1.80
依存関係の探索	0.80
関数呼出しグラフの生成	205.18
EOL と判定された OSS の特定	0.32
脆弱性影響範囲の特定	6.52
到達可能なレガシー関数の特定	752.35
ソフトウェアリスク特定結果出力	0.02
合計	966.99

において，ソフトウェアリスクを特定可能なことを示した．実現方式における処理として，ソースコードに対し関数単位の最終更新日時の取得処理に主要な時間を要している．関数ごとの更新履歴を取得するために，`GitHub API` へ繰り返し問い合わせしており，ネットワーク通信および API の応答に時間を要するためである．

6.2 提案手法の考察

提案手法では，依存関係と被依存関係を把握し，CVSS，EOL と判定された OSS，脆弱性影響範囲，到達可能なレガシー関数を特定し，ソフトウェアリスクを提示可能とした．

脆弱性影響範囲と到達可能なレガシー関数の特定では，直接呼出しによる呼出し関係を関数呼出しグラフから特定可能なことを確認した．到達可能なレガシー関数は，エントリ関数から到達可能な関数に対し関数ごとの最終更新日時を取得することで特定可能である．EOL と判定された OSS の特定では，EOL 情報提供サービスに対象 OSS のバージョン情報が存在せず，照合できない場合があり，OSS 開発者が公開する EOL 情報の参照が必要と考えている．

6.3 適用対象 OSS の考察

提案手法の実現方式は C 言語を対象とした．他のプログラミング言語への適用可能性として，各プログラミング言語に対応した関数呼出しグラフ生成手法が必要となる．また，レガシー関数の特定に用いる関数の最終更新日時は `GitHub Blame` から取得しているため，GitHub 上で管理される OSS が適用対象である．

提案手法では，GitHub 上の更新履歴を用いるため，ソフトウェアシステムで利用される OSS に独自の変更が加えられた場合，ソースコードの更新状況を反映できない．独自に変更された OSS に対しては，ローカル `git` レポジトリにて管理され，`blame` コマンドを利用して情報取得可能とする必要がある．

6.4 関数呼出しの限界

提案手法の実現方式では，関数呼出しグラフにおける呼出し関係として直接呼出しのみを対象としており，関数ポインタを介した間接呼出しには対応していない．実際には呼出し関係が存在する場合でも関数呼出しグラフ上では呼出し関係を特定できない可能性がある．

関数ポインタ対応として，バイナリ解析を用い，間接呼出しを含む呼出し関係を補完し，実際の呼出し関係を正確に反映した関数呼出しグラフを生成する必要がある．

7. 関連研究

OSS 管理者の脆弱性管理調査

Jessy らは，GitHub Advisory Database に掲載されたプロジェクトを管理する OSS 管理者に対し脆弱性管理上の課題調査を実施した [16]．アンケート調査とインタビュー調査から，OSS 管理において，脆弱性と大量の依存関係から，脆弱性管理に要する作業負担の増大が示されている．

セキュリティ修正未反映のソースコード

Alfusainey らは，プログラマ間のソースコード共有プラットフォームである Stack Overflow 上で再利用されたコードスニペットが，適切にセキュリティの修正を実施しているかを更新履歴から特定した [17]．Stack Overflow 上のソースコードは，時間が経過していても脆弱性が発見される場

[*1] 対象 OSS の CVE 修正前の GitHub レポジトリからレガシー情報を取得するため，同一 deb パッケージにおいてもレガシー関数の割合は異なる．

© 2026 Information Processing Society of Japan

表 5　既存研究と提案手法の比較

項目	VPChecker[2]	Alfusainey ら [17]	提案手法
解析形式	バイナリ	ソースコード	ソースコード
解析対象	Debian	Stack Overflow	Debian
着目情報	脆弱性情報	コード更新履歴	脆弱性・レガシー・EOL 情報
特定対象	影響範囲	セキュリティ修正未反映コード	ソフトウェアリスク

合が存在するため，継続的な監視が必要だとしている．

脆弱性影響範囲の特定と到達可能性解析

VPChecker は，バイナリコードでの脆弱な関数への到達可能性を調査する脆弱性影響特定手法を提案している [2]．VPChecker では，脆弱な関数への到達可能性がない場合，脆弱性の影響を受けないことを実証した．

7.1 比較

提案手法と既存研究の比較を表 5 に示す．

VPChecker は C/C++ バイナリファイルから関数呼出しグラフを生成し，依存関係を基に，脆弱な関数への到達可能性を特定している．また，Alfusainey らは，プログラマ間のソースコード共有プラットフォームである Stack Overflow から，再利用されたソースコードの更新履歴を取得し，未反映のセキュリティ修正を特定している．

本研究は，既存研究で扱われている脆弱性影響範囲に加え，関数の最終更新履歴に基づくレガシー関数の到達可能性を新たに考慮した．提案手法にて，OSS の依存関係情報，被依存関係情報とソースコード解析により関数呼出しグラフを生成し，脆弱性影響範囲，到達可能なレガシー関数，および EOL と判定された OSS を特定可能と確認した．

8. まとめ

多様な OSS を利用するソフトウェアシステムには，安定運用と迅速なセキュリティ対応が求められる．既存の脆弱性影響範囲の特定手法は，開発過程で発生するレガシー関数を含む長期間更新されていないソースコードや，サポート終了の OSS との依存関係を捉えられない課題がある．

本研究は，ソフトウェアリスクの特定のために，既存の脆弱性影響範囲の特定手法 [1] を拡張し，依存関係にある OSS の EOL 情報，ならびにレガシー情報を用い，OSS の依存関係と被依存関係に基づいた関数粒度での到達可能性を考慮したソフトウェアリスクの特定手法を提案した．

評価として，提案手法により，脆弱性が報告された OSS，依存関係および被依存関係を有する OSS のソースコードを静的解析し，関数呼出しグラフを生成し，CVSS，EOL と判定された OSS，脆弱性影響範囲，および到達可能なレガシー関数を特定し，ソフトウェアリスクを 3 段階で判定し，一定時間において処理可能なことを示した．

謝辞　本研究の一部は，JSPS 科研費 JP26K21217，JST ACT-X JPMJAX24M4，ならびに経済安全保障重要技術育成プログラム JPMJKP24K2 の支援を受けたものである．

参考文献

[1] 葛野弘樹，矢野智彦，面　和毅：OSS の依存関係を利用したソースコード解析による脆弱性影響範囲特定手法の提案，第 74 回情報通信システムセキュリティ研究会 (2026).

[2] Patel, H., Snit, A. and Polychronakis, M.: Supply Chain Reaction: Enhancing the Precision of Vulnerability Triage using Code Reachability Information, *2025 IEEE Annual Computer Security Applications Conference*, pp. 92–107 (2025).

[3] Hu, Y., Li, C., Wu, Y., Cai, Y. and Zou, D.: VulSCA: A Community-Level SCA Approach for Accurate C/C++ Supply Chain Vulnerability Analysis, *Network and Distributed System Security Symposium 2026* (2026).

[4] Svensson, N.: Defining and Identifying Legacy Code in Software, A case study developing 3D visual camera surveillance software, Master's thesis, Malmö University, Faculty of Technology and Society (2018).

[5] endoflife.date: endoflife.date, (online), available from ⟨https://endoflife.date/⟩ (accessed 2026-08-10).

[6] CVE Program: CVE – Common Vulnerabilities and Exposures, (online), available from ⟨https://www.cve.org/⟩ (accessed 2026-08-14).

[7] FIRST: Common Vulnerability Scoring System v3.1: Specification Document, (online), available from ⟨https://www.first.org/cvss/v3.1/specification-document⟩ (accessed 2026-08-15).

[8] 情報処理推進機構 (IPA)：共通脆弱性評価システム CVSS v3 概説，情報処理推進機構 (IPA) (オンライン)，入手先 ⟨https://www.ipa.go.jp/security/vuln/scap/cvssv3.html⟩ (参照 2026-08-19).

[9] Debian Project: Debian Policy Manual, (online), available from ⟨https://www.debian.org/doc/debian-policy/⟩ (accessed 2026-08-10).

[10] debsecan: debsecan - the Debian Security Analyzer, Debian Project (online), available from ⟨https://wiki.debian.org/DebianSecurity/debsecan⟩ (accessed 2026-08-14).

[11] debsbom: debsbom - SBOM generator for Debian-based distributions, Siemens AG (online), available from ⟨https://github.com/siemens/debsbom⟩ (accessed 2026-08-14).

[12] Debian Project: apt(8), (online), available from ⟨https://manpages.debian.org/trixie/apt/apt.8.en.html⟩ (accessed 2026-08-19).

[13] GNU cflow: GNU cflow, GNU Project (online), available from ⟨https://www.gnu.org/software/cflow/⟩ (accessed 2026-08-14).

[14] Git Project: git-blame Documentation, Git Project (online), available from ⟨https://git-scm.com/docs/git-blame/ja.html⟩ (accessed 2026-08-12).

[15] National Vulnerability Database: CVE-2026-4176, National Institute of Standards and Technology (online), available from ⟨https://nvd.nist.gov/vuln/detail/CVE-2026-4176⟩ (accessed 2026-08-19).

[16] Ayala, J., Tung, Y.-J. and Garcia, J.: A Mixed-Methods Study of Open-Source Software Maintainers On Vulnerability Management and Platform Security Features, *34th USENIX Security Symposium*, pp. 2105–2124 (2025).

[17] Jallow, A., Schilling, M., Backes, M. and Bugiel, S.: Measuring the Effects of Stack Overflow Code Snippet Evolution on Open-Source Software Security, *2024 IEEE Symposium on Security and Privacy*, pp. 1083–1101 (2024).

© 2026 Information Processing Society of Japan